



Security Audit & Vulnerability Report

FrantzCoutard.com — Student Challenge Platform

OVERALL RISK: CRITICAL

PHP 8.2 API + React / Vite Frontend

Scope: complete codebase (`api/` + `frontend/`)

Methodology: OWASP Top 10 · CWE · SANS Top 25

Date: 9 July 2026

Security Score: **41 / 100**

Table of Contents

- 1. Executive Summary
- 2. Scores & Issue Breakdown
- 3. Findings by Severity (index)
- 4. Detailed Findings — Critical
- 5. Detailed Findings — High
- 6. Detailed Findings — Medium
- 7. Detailed Findings — Low
- 8. Positive Controls
- 9. Remediation Roadmap
- 10. Top 10 Fixes · Quick Wins · Long-Term
- 11. Final Summary & Conclusion

1. Executive Summary

The platform is **well-engineered in several ways**: SQL is almost entirely parameterized (no SQL injection found), the React frontend has no unsafe HTML sinks (no XSS found), session IDs regenerate on login, CSRF is enforced with a strong token, CORS is not wildcarded, and store prices are re-derived server-side.

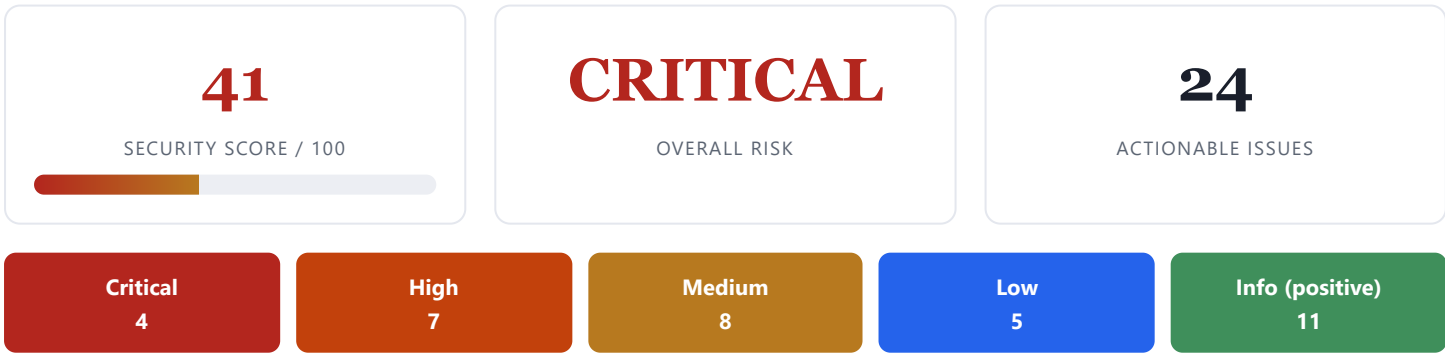
However, the audit found **serious, exploitable problems that must be fixed before going live**. The two most urgent were **verified live** against the running server:

- **A hardcoded admin backdoor** — a demo admin account with a publicly documented password logs in via the normal login form and grants full admin, even when "demo mode" is off.
- **The secrets file is downloadable** — `GET /api/.env` returned HTTP 200 with contents, exposing the Google OAuth client secret + refresh token.

Also: the **Razorpay/PayPal payment verification can be replayed** to mark expensive orders paid, there is **no rate limiting anywhere**, **account takeover via public re-registration** is possible, and several **access-control gaps** let the wrong user write another user's data.

Verdict: Not safe to expose publicly until the Critical + High items are remediated. Most fixes are small; the emergency items can be done in under an hour.

2. Scores & Issue Breakdown



3. Findings by Severity (index)

ID	Finding	Severity
C-1	Hardcoded demo admin backdoor usable via normal login	CRITICAL
C-2	<code>api/.env</code> secrets downloadable over HTTP	CRITICAL

C-3	Razorpay payment verification replay / order-not-bound	CRITICAL
C-4	PayPal capture — order-not-bound / no amount check	CRITICAL
H-1	Account takeover via public re-registration	HIGH
H-2	IDOR write — any role overwrites any student's interview	HIGH
H-3	No rate limiting (brute-force / enumeration / flood)	HIGH
H-4	Stock drained without payment	HIGH
H-5	Unauthenticated 70 MB upload (new-school/upload)	HIGH
H-6	APP_DEBUG=true leaks exceptions / DB errors	HIGH
H-7	Weak password policy (6-char min)	HIGH
M-1	Student can self-award winner / score / rank	MEDIUM
M-2	Judge can view/score ANY submission	MEDIUM
M-3	Stripe confirm doesn't re-verify amount/currency	MEDIUM
M-4	Email verification globally bypassed	MEDIUM
M-5	No security headers / no HTTPS enforcement	MEDIUM
M-6	User enumeration (register / forgot-password)	MEDIUM
M-7	editor role treated as full admin	MEDIUM
M-8	Uploads dir has no script-execution block	MEDIUM
L-1...L-5	Student PII lookup · gov-id URL · session timeout · test files · Vite bump	LOW

4. Detailed Findings — Critical

C-1 — Hardcoded demo admin backdoor usable via normal login

Verified:  live — login as `demo.admin@frantzcoutard.demo / demo1234` → admin; `GET admin/submissions` → 200

Files: `api/lib.php` (`demo_user / demo_ensure_accounts` ~3713–3890); `api/index.php` (POST `auth/login` ~125–173) · **CWE-798, CWE-912**

Description: Demo accounts (Admin, Sponsor, ...) are all seeded with the same public password `demo1234`, shown on the `/demo` page. The "demo mode" switch only hides the one-click buttons — it does not stop these accounts logging in through the normal login form.

Why dangerous: Anyone with the (public) credentials gets full admin permanently, even after `DEMO_MODE=off`.

Attack: Read `demo1234` from the demo page/source → log in as `demo.admin@...` → full admin (edit content, approve users, impersonate anyone, read all data).

Fix: Only create demo accounts when demo mode + local env; give them **random** passwords (login only via the demo endpoint); on `DEMO_MODE=off` delete/disable them and reject `@frantzcoutard.demo` logins.

```
// secure: unknowable password
$hash = password_hash(bin2hex(random_bytes(24)), PASSWORD_DEFAULT);
```

C-2 — `api/.env` secrets downloadable over HTTP

Verified:  live — `GET /api/.env` → HTTP 200 + file body

Files: `api/.env` (`GOOGLE_CLIENT_SECRET`, `GOOGLE_REFRESH_TOKEN`, mail creds); `api/.htaccess` (no dotfile deny) · **CWE-538, CWE-312**

Description: The secrets file sits in the web-served `api/` folder and no rule blocks it; the front-controller rewrite skips real files, so Apache serves `.env` directly. (It is correctly git-ignored — not committed — but still web-served.)

Why dangerous: Google OAuth client secret + refresh token = send mail as you / abuse Google APIs; DB config revealed.

Attack: `curl https://site/api/.env` → download all secrets.

Fix (now): Deny dotfiles in `api/.htaccess` and **rotate** the exposed secrets immediately. Prefer moving `.env` outside the web root.

```
<FilesMatch "^\.env">
    Require all denied
</FilesMatch>
```

C-3 — Razorpay verification replay / order-not-bound

Files: `api/index.php` (`store/checkout/razorpay-verify` ~1197–1212); `api/lib.php` (`storefront_mark_order_paid`) · **CWE-345, CWE-639**

Description: The signature is verified correctly, but the server never checks that `razorpay_order_id` is the one it created for `order_no`, nor that the paid amount equals the order total. It marks the client-supplied `order_no` paid.

Why dangerous / Attack: Pay for one \$1 item (get a valid signature), then replay verify with a different/expensive `order_no` (guessable `FC-#####`) → that order flips to paid without payment.

Fix: Load the order, require `order.payment_session_id === razorpay_order_id`, fetch the payment from Razorpay and assert `amount+currency == order total`, and enforce a UNIQUE constraint on `payment_intent_id`.

C-4 — PayPal capture not bound to order / no amount check

Files: `api/index.php` (`store/checkout/paypal-capture` ~1215–1228) · **CWE-345, CWE-639**

Description / Attack: Same class as C-3 — the capture status is checked, but the `paypal_order_id` isn't tied to `order_no` and the captured amount/currency isn't compared to the order total. Pay once → mark a different order paid; underpayment accepted.

Fix: Verify `order.payment_session_id === paypal_order_id`; assert capture `amount.value / currency_code == order total`; enforce `payment_intent_id` uniqueness.

5. Detailed Findings — High

H-1 — Account takeover via public re-registration

Files: `api/new_school_routes.php` (`new_school_upsert_user_account` ~5–46) via all public register routes · **CWE-620, CWE-287**

Description: Registering with an **existing non-admin email** overwrites that account's `password_hash` and `role` with attacker values and logs them in — no check that the email is already verified/owned.

Attack: `POST new-school/student/register` with `email=victim@...`, chosen password → victim's account taken over.

Fix: If the existing account has `email_verified_at` set, reject with 409 (mirror `auth/register`); never reset password/role during registration.

H-2 — IDOR write: any role can overwrite any student's business interview

Files: `api/new_school_routes.php` (`POST new-school/business` ~2316–2388) · **CWE-639**

Description: The handler blocks teacher/school and handles student/parent, but every other approved role (judge/business/sponsor/media/volunteer/member) hits an `elseif ($studentId > 0)` branch that trusts an arbitrary `student_id` with no admin/ownership check.

Fix: Restrict that branch to admin roles, exactly like `POST new-school/submission` does.

H-3 — No rate limiting anywhere

Files: `api/index.php` (`auth/login` ~125, `register` ~34, `admin-login` ~242, `forgot-password` ~175); `payment-verify` + `upload` routes · **CWE-307**

Description / Attack: No throttling/lockout/CAPTCHA. Unlimited password brute-force, `payment-verify` hammering (amplifies C-3/C-4), `upload` flooding (H-5).

Fix: Per-IP + per-account rate limiting with backoff + lockout; CAPTCHA after N failures.

H-4 — Inventory drained without payment

Files: `api/index.php` (`store/checkout` ~1050; `POST order` ~1230–1310) · **CWE-840**

Description: Stock is decremented when checkout starts (before payment) with no restoration on abandonment, and `POST order` confirms + decrements with no payment step at all.

Fix: Reserve stock with TTL or decrement only on confirmed payment; release stock for expired pending orders; remove/convert the unpaid `POST order` route.

H-5 — Unauthenticated 70 MB file upload

Files: `api/new_school_routes.php` (`POST new-school/upload` ~2723–2779) · **CWE-434, CWE-770**

Description: No auth guard — anyone can upload 70 MB videos / documents. (Naming is safe, so not RCE.) Enables disk-exhaustion DoS and hosting arbitrary files on your domain.

Fix: Require login/enrollment token + throttling + daily quota.

H-6 — APP_DEBUG=true leaks exceptions / DB errors

Files: `api/.env` (`APP_DEBUG`); `api/config.php` (~50); `api/index.php` (global catch ~2778) · **CWE-209** · *manual verification on prod host*

Description / Fix: With debug on, raw exception + DB messages are returned to clients. Force `APP_DEBUG=false` in production; log server-side only.

H-7 — Weak password policy (6-char minimum) HIGH

Files: api/index.php (~41, ~215, ~337) · **CWE-521** · **NIST SP 800-63B**

Description / Fix: Only 6 chars, no complexity/breach check, even for admins. Raise to 12+, check against a breached-password list, enforce for privileged roles. Pairs dangerously with H-3.

6. Detailed Findings — Medium

ID	Finding & File	Why / Fix
M-1	Student self-awards winner/score/rank — <code>manage/submission</code> ~3930-4005 (CWE-639)	status/score/rank read from body with no privilege gate. Fix: accept these only for admin/teacher scope; students limited to draft/submitted.
M-2	Judge can view/score ANY submission — <code>new_school_judge_can_review</code> ~3959 (CWE-639) (<i>verify intended model</i>)	Guard returns true when no assignment row exists. Fix: require present non-recused assignment, or remove misleading "not assigned" text.
M-3	Stripe confirm no amount re-check — <code>store/checkout/confirm</code> ~1106 (CWE-345) (<i>low exploitability</i>)	Binds <code>order_no</code> + requires paid (good); also assert <code>amount_total</code> /currency == order total.
M-4	Email verification globally bypassed — <code>api/index.php</code> ~33/67/146 (CWE-287)	Register/login force <code>email_verified_at=NOW()</code> . Re-enable OTP verification before prod; gate bypass behind an env flag off in prod.
M-5	No security headers / HTTPS — <code>config.php</code> + <code>.htaccess</code> (CWE-693)	No CSP/X-Frame-Options/HSTS/X-Content-Type-Options; cookie <code>secure</code> only on HTTPS. Add headers + force HTTPS + HSTS.
M-6	User enumeration — register ~48 / forgot-password ~183 (CWE-204)	Responses reveal whether an email exists. Fix: uniform generic responses.
M-7	<code>editor</code> == full admin — <code>require_admin</code> ~152 (CWE-269)	Editors reach all admin routes incl. impersonation. Fix: separate editor capability tier.
M-8	Uploads dir no execution block — <code>api/uploads/</code> (CWE-434, defense-in-depth)	Add <code>.htaccess</code> disabling PHP engine + denying executable extensions.

7. Detailed Findings — Low

ID	Finding	File · Fix
L-1	Enumerable unauthenticated student lookup (minor PII: name/school/teacher) via 8-digit participant_id	<code>new_school_routes.php</code> ~813-852 · require high-entropy <code>qr_token</code> + rate-limit (<i>verify: intended public page?</i>)
L-2	<code>government_id_url</code> stored without format validation	~1138/1231 · validate URL prefix; render safely in admin UI
L-3	No session idle/absolute timeout	<code>config.php</code> ~15-26 · add expiry for privileged sessions
L-4	Leftover <code>shell-write-test.txt</code> / <code>testx.txt</code> in public upload dir	delete; confirm origin
L-5	Vite 5.4.8 (dev-server advisories)	<code>frontend/package.json</code> · bump latest 5.4.x (dev-only impact)

8. Positive Controls (keep these)

- No SQL injection — parameterized PDO / int casts / whitelists; `ATTR_EMULATE_PREPARES=false`.
- No frontend XSS sinks — no `dangerouslySetInnerHTML` / `innerHTML` / `eval`; React auto-escaping.
- Session fixation prevented — `session_regenerate_id(true)` on login.
- Strong CSRF — `random_bytes(32)` token, `hash_equals`, enforced on non-GET.
- CORS not wildcard — echoed only on exact configured-origin match.
- Server-side price integrity — client prices ignored, catalog re-read.
- Safe upload naming — content MIME + server-forced extension + random name.
- Oversell prevented — `SELECT ... FOR UPDATE` row locks.
- No dynamic code execution / backdoors — `escapeshellarg` on shell; no `eval`.

- Secrets not committed — `.env` git-ignored; example has placeholders only.
- Stripe confirm binds order (`client_reference_id == order_no`) + idempotent.

9. Remediation Roadmap

Phase	Actions
 Phase 0 — today	C-2 deny <code>.env</code> + rotate Google/mail secrets · C-1 kill demo backdoor · H-6 <code>APP_DEBUG=false</code>
 Phase 1 — this week	C-3/C-4 bind provider order + verify amount · H-1 block re-registration overwrite · H-2/M-1/M-2 access-control fixes · H-3 rate limiting · H-5 auth upload · H-4 stop unpaid stock drain
 Phase 2 — this month	H-7/M-4 stronger passwords + email verification · M-5 security headers + HTTPS · M-6 enumeration-safe responses · M-7 editor tier · M-8 uploads exec block
 Phase 3 — ongoing	Low items · session timeouts · dependency bumps · SAST + dependency scanning in CI · payment webhooks as source of truth

10. Top 10 Fixes · Quick Wins · Long-Term

Top 10 highest priority	Quick wins (low effort)	Long-term
1. Remove demo admin backdoor (C-1) 2. Block <code>.env</code> + rotate secrets (C-2) 3. Fix Razorpay verify (C-3) 4. Fix PayPal capture (C-4) 5. Stop account-takeover re-registration (H-1) 6. Rate limiting (H-3) 7. <code>APP_DEBUG=false</code> (H-6) 8. Fix IDOR business-interview (H-2) 9. Authenticate 70 MB upload (H-5) 10. Fix self-awarded winner/score (M-1)	• <code>.env</code> deny rule — 3 lines • <code>APP_DEBUG=false</code> — 1 setting • <code>uploads/.htaccess</code> exec block — 5 lines • Security headers block — few lines • <code>require_login()</code> on upload — 1 line • Restrict <code>elseif (\$studentId>0)</code> to admins — 1 condition	• Rate-limiting / WAF layer • Payment webhooks (signed) as source of truth • Email verification + MFA for admins • Capability/permission model (replace flat editor==admin) • SAST + dependency scanning in CI

11. Final Summary & Conclusion

Metric	Count
Total actionable issues	24
 Critical	4
 High	7
 Medium	8
 Low	5
 Informational (positive controls)	11
Security Score	41 / 100
Overall Risk	CRITICAL

Conclusion: The application shows good fundamentals (parameterized SQL, clean React output, CSRF, session regeneration, server-side pricing) but carries **four Critical issues** — a hardcoded admin backdoor, a downloadable secrets file, and two payment-verification bypasses — plus authentication and access-control weaknesses. **Do not expose this platform publicly until Phase 0 and Phase 1 are complete.** Most fixes are small and well-scoped; the emergency items (C-2, C-1, H-6) can be done in under an hour.

This report reflects the codebase at 2026-07-09. Items marked "manual verification" should be confirmed on the production host. Re-audit after remediation. Two Critical findings (C-1, C-2) were verified live against the running server.